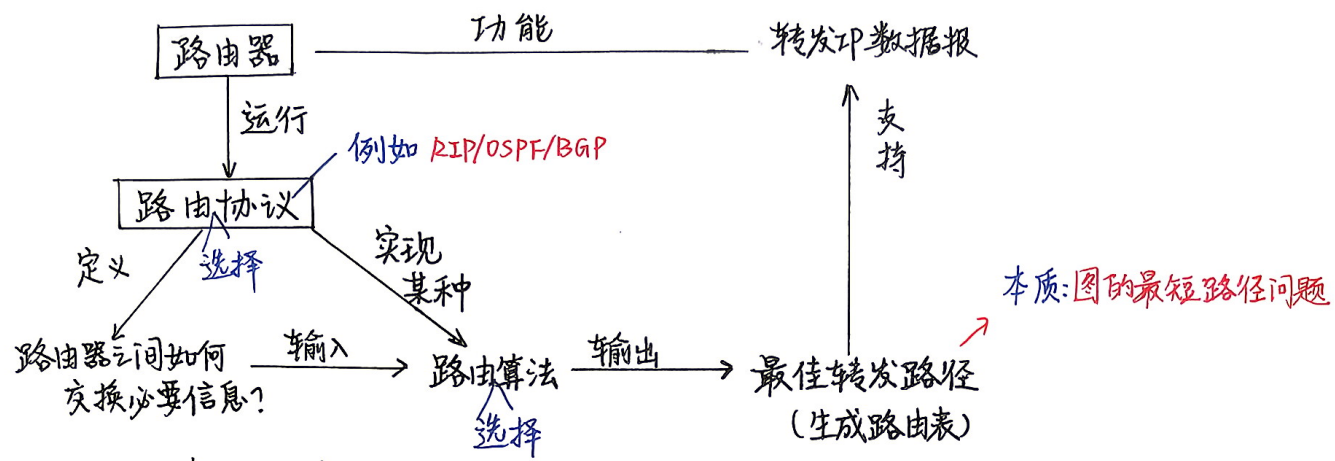
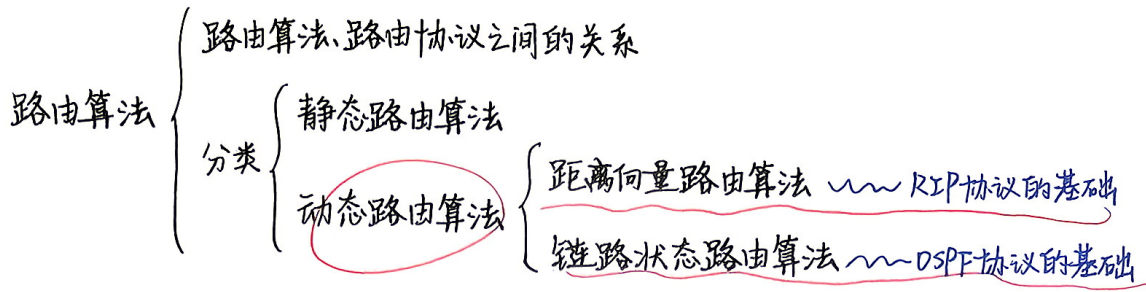
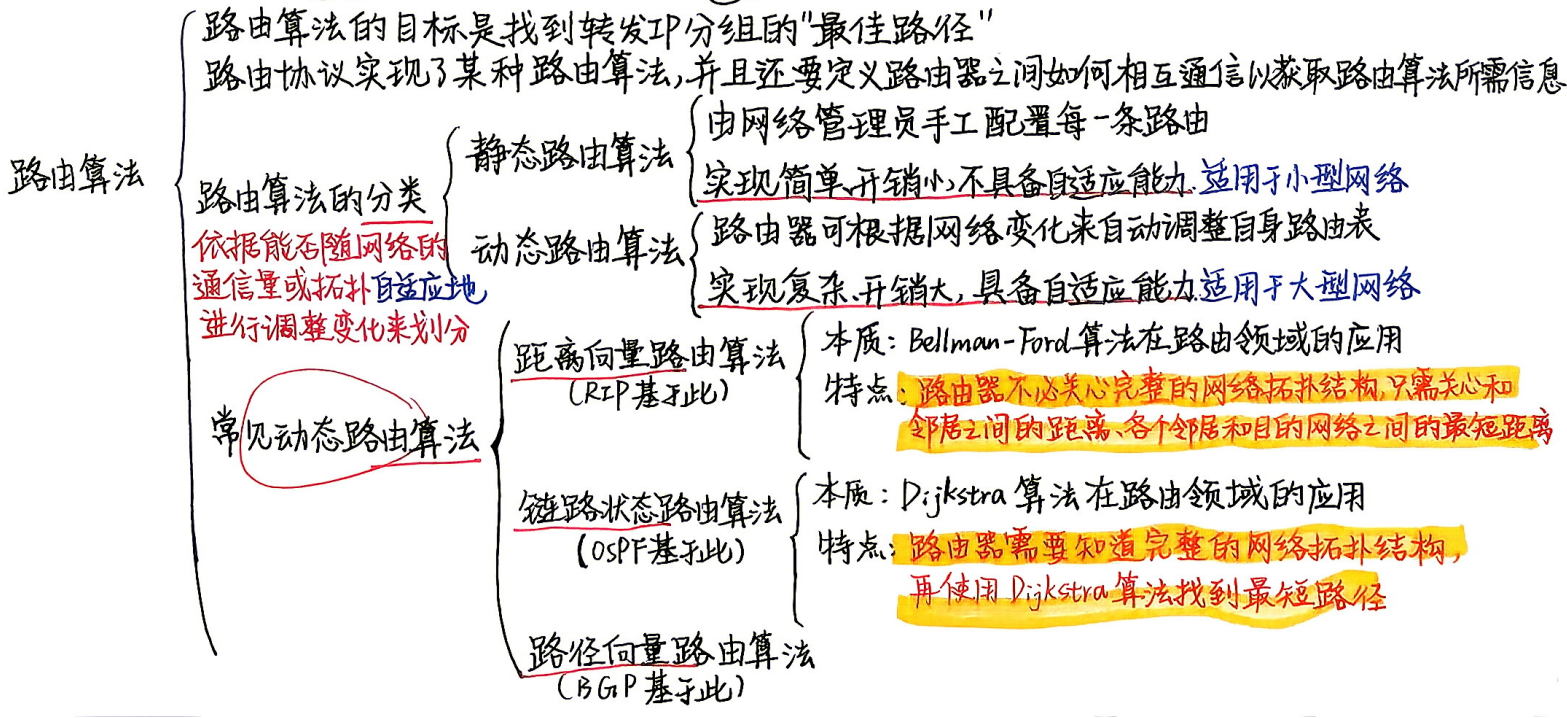
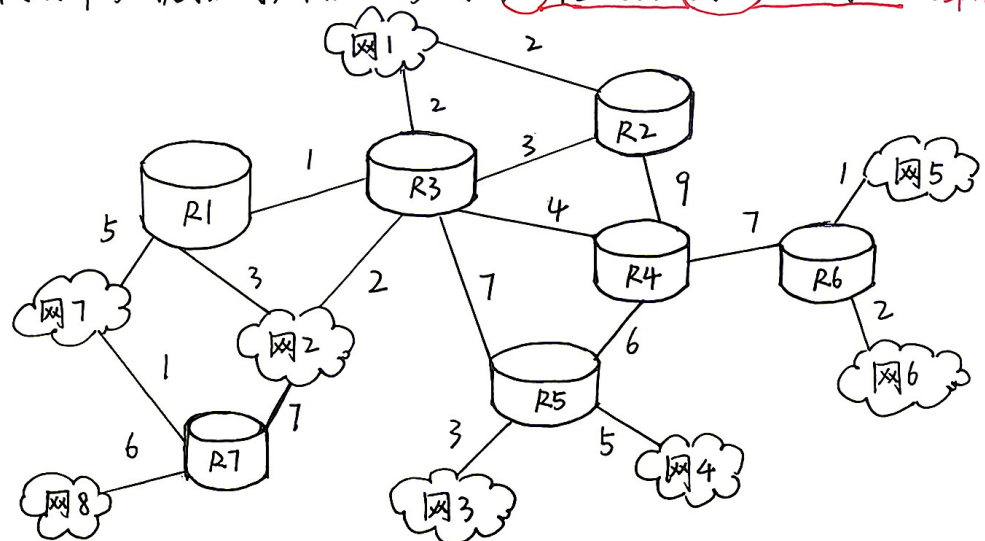


路由算法



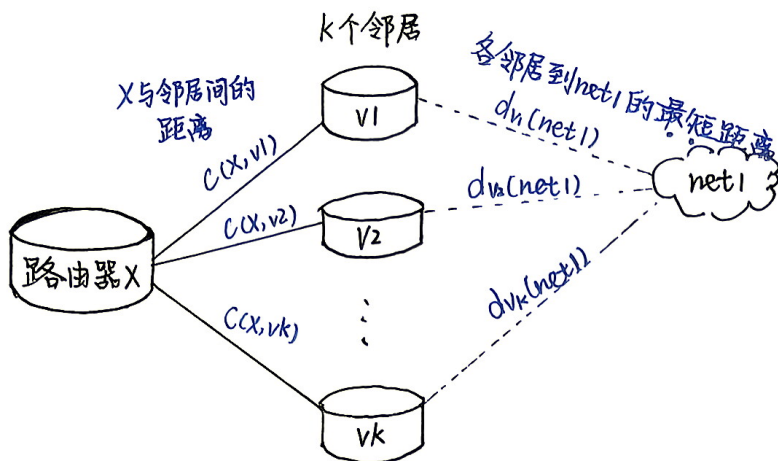
路由协议会制定一种规则,度量一条边的权值/代价

当一台路由器转发IP数据报时,本质就要找到从本路由器到达目的网络的“最优路径”



距离-向量路由算法

一片区域内有一个目的网络:

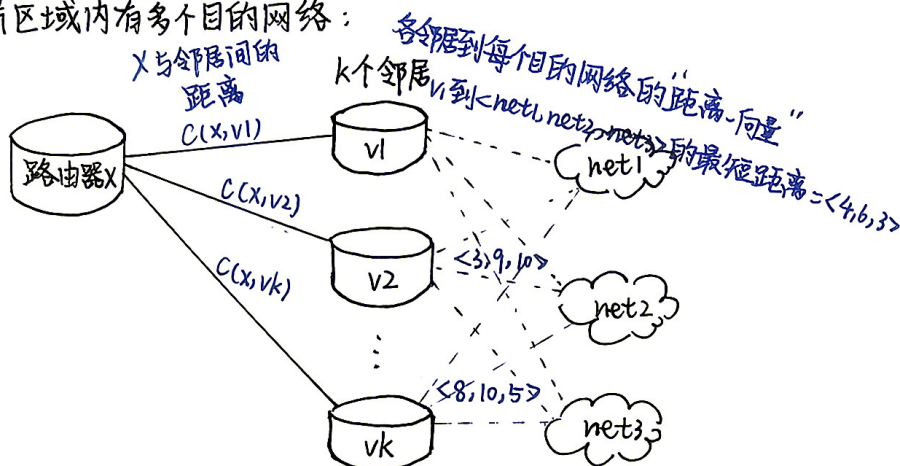


Bellman-Ford算法的核心思想:

假设 $d_X(net1)$ 表示从节点 X 到节点 net1 的最短距离 (最小代价), 则有 $d_X(net1) = \min \{C(X, V_i) + d_{V_i}(net1)\}$ 其中, V_i 是 X 的所有邻居

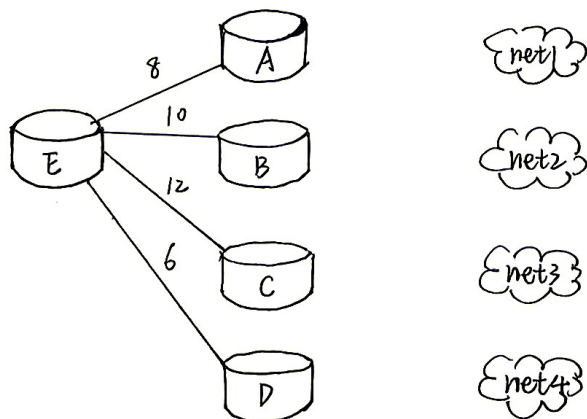
不同路由协议对"距离"的定义可能不同:
规定直接相连的两个节点之间距离等于1
或
以往返时延作为链路的距离

一片区域内有多个目的网络:



例. 某网络中所有路由器均采用距离向量路由算法计算路由, 由路由器 E 与邻居路由器 A, B, C, D 之间的直接链路距离分别是 8, 10, 12, 6, 且 E 收到邻居路由器的距离向量如下表所示. 则路由器 E 更新后的到达目的网络 Net1 ~ Net4 的距离分别是?

目的网络	距离向量				
	A 的	B 的	C 的	D 的	
Net1	$1+8$	$23+10$	$20+12$	$22+6$	9
Net2	$12+8$	$35+10$	$30+12$	$28+6$	20
Net3	$24+8$	$18+10$	$16+12$	$36+6$	28
Net4	$36+8$	$30+10$	$8+12$	$24+6$	20



答案: 9, 20, 28, 20

链路状态路由算法

要求每台路由器节点了解完整的网络拓扑结构(全网共?节点, ?节点相连, 代价?, ...)

只要一台路由器知道完整的网络拓扑结构(数据结构: 带权图), 就可以用Dijkstra算法转到最优转发路径.

由于:

RIP 路由协议: 基于距离向量路由算法, 路由器要把自身的距离向量告诉邻居们.

网络数量越多, 距离向量越大

N 个 $\rightarrow$ N 维

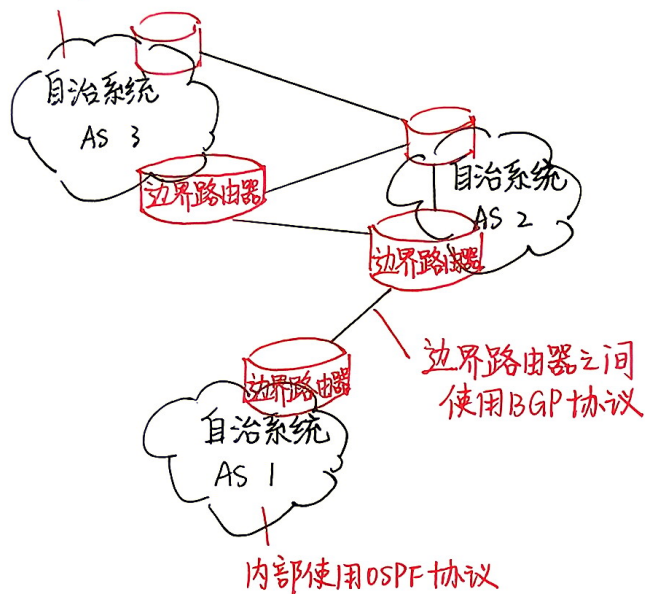
OSPF 路由协议: 基于链路状态路由算法, 要求每一台路由器都要建立整个网络的拓扑图

全世界的路由器数量、网络数量以亿计, 不可能让每台路由器都知晓全世界的网络情况.

因此:

RIP、OSPF 不能用于全世界范围的路由器

内部使用 RIP 协议



边界路由器不关心在另一自治系统内部.

网络被拆分成多少子网.

只需关心整个自治系统对外暴露的 CIDR 地址块.

转发 IP 数据报时只要目的网络号

能与这个自治系统对外展现的 路由聚合 得来

CIDR 地址块对上, 就往该自治系统的边界路由器转发即可.

分层次的路由协议

"网关" 即 "路由器", 早期术语

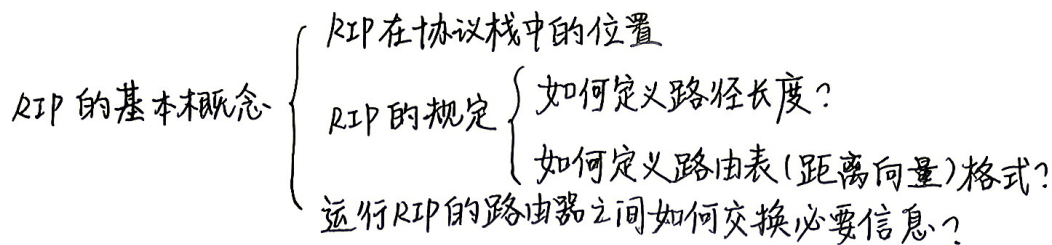
将全世界的网络划分为多个相互独立的 自治系统.

- AS 的管理单位有权决定在本自治系统内, 使用何种 内部路由协议 (如 RIP、OSPF)
- 每个 AS 至少要有 一台自治系统边界路由器 与其它 AS 相连
- 各边界路由器之间使用统一的 外部路由协议 (如 BGP-4)
- 自治系统 之间 的路由选择亦称 域间路由选择, 自治系统 内部 的路由选择亦称 域内路由选择
- 每个自治系统拥有全球唯一的 AS 编号 (ASN)
- 自治系统之间是平级关系, 不存在包含关系 (意味着 "分层" 其实也就是两层)
- 一个自治系统常包含一个或多个 CIDR 地址块 (便于路由聚合)

路由协议的分类

- | | | | | |
|---------|---|--------------|---|-------------------------------------|
| 路由协议的分类 | { | 内部网关协议 (IGP) | { | 用于自治系统 <u>内部</u> 的路由选择 (如 RIP、OSPF) |
| | | 外部网关协议 (EGP) | | 用于自治系统 <u>之间</u> 的路由选择 (如 BGP) |
- 一个自治系统有权决定其内部使用何种内部网关协议.
- 各个自治系统的边界路由器之间统一使用 BGP 协议相互通信路由信息.

路由信息协议 RIP



RIP 在协议栈中的位置

RIP 属于应用层, 使用 UDP 传递数据 (端口 = 520)

RIP 协议定义了 2 种报文:

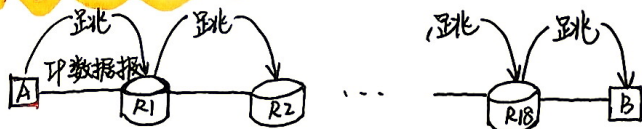
- 请求 (Request) 报文:
请求邻居路由器发送其路由表, 用于启动或查询
- 响应 (Response) 报文:
响应请求或定期发送给邻居路由器, 携带完整或部分路由表

每个 Response 报文最多携带 25 个路由表项信息. 如果路由表太大, 就要拆分为多个 Response 报文, 每个报文携带部分路由表

RIP 的规定

① 路径长度?

- RIP 使用 **跳数** (亦称距离) 来衡量到达目的网络的距离.
规定: 路由器到直连网络的距离为 1, 而每经过一个路由器, 距离加 1
- RIP 认为好的路由就是跳数最少的
- RIP 允许一条合法路径距离不能超过 15. 距离等于 16 时表示网络不可达
⇒ RIP 只适用于小型自治系统



若超过 15 时, 到 R1 检测发现剩余距离为 15 或以上, 则判定不可达, 丢弃 IP 数据报

② 路由表格式?

- 每个路由器都要维护自己的路由表, 路由表项有 3 个关键字段:

<目的网络 N, 距离 d, 下一跳路由器地址 X>

用 CIDR 记法表示
某网络的
网络前缀是多少

目的网络	距离	下一跳路由器
Net1	1	直接交付
Net2	16	NULL
Net3	2	R6
Net4	2	R9

IP 地址表示

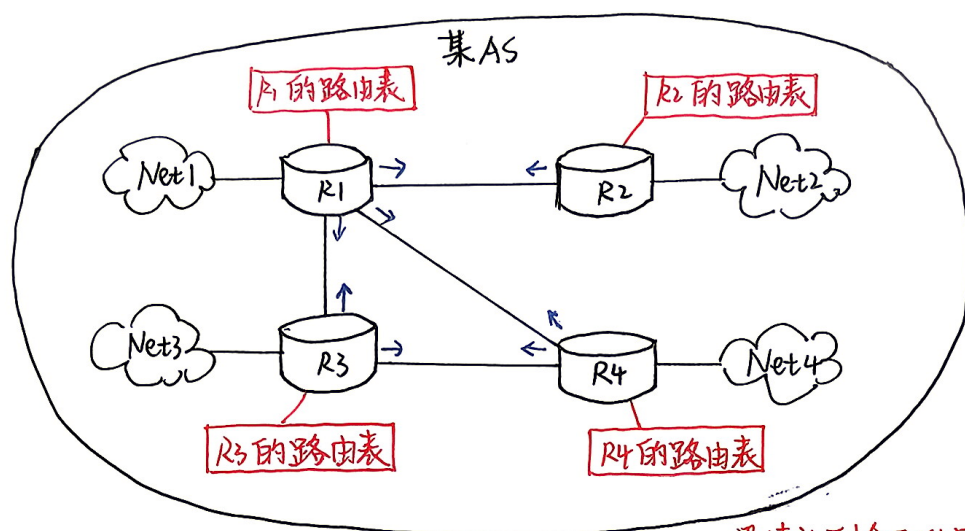
- 每个路由器都要维护从它自身到其它每个目的网络的距离记录, 即距离向量 (距离向量信息包含在路由表中, RIP 使用 "距离向量路由算法" 计算最短路径.)

运行RIP的路由器之间如何交换路由算法所需的必要信息？

Who? 仅与直接相邻的路由器交换信息

What? 交换的信息是本路由器所知道的全部信息，即自身路由表

When? 通常按固定时间间隔（通常为30秒）交换路由信息



注：为加快RIP收敛速度，可以引入触发更新机制，当路由器发现网络拓扑变化时，立即向相邻路由器通告拓扑变化后的路由信息，而不必等待30秒。
→ 迅速达到全局的同步
逐步生成

每个路由器每隔固定时间就会给自己的邻居发送自己的路由表。

而路由表被封装在RIP响应报文中。

↓
其实就是把自己到达各个网络的距离向量告知邻居

那么对于一台路由器而言，只要它定期收到来自各个邻居的距离向量，就可通过“距离-向量”

路由算法动态寻找最优的转发路径。

RIP的工作过程(基于距离-向量算法)

路由器刚开始工作时,只知道自己直接相连的几个网络的距离为1.

每个路由器仅与相邻路由器周期性地交换并更新路由信息. 经过若干次的交换和更新后,所有的路由器最终都会知道到达本自治系统内任何网络的最短距离和下一跳路由器的地址,称为收敛

对每个相邻路由器发来的RIP报文,执行如下步骤:

① 对地址为X的相邻路由器发来的RIP报文,先修改该报文中的所有项目:
把“下一跳”字段中的地址都改为X,并把所有距离字段的值加1

本质:
距离向量路由算法

② 对修改后的RIP报文中的每个项目执行如下步骤:

IF (原路由表中没有目的网络N)

则将该项目添加到路由表中(表明这是新的目的网络)

ELSE IF (原路由表中有目的网络N且下一跳路由器的地址是X)

则用收到的项目替换原路由表中的项目(要以更新的消息为准)

ELSE IF (原路由表中有目的网络N且下一跳路由器的地址不是X)

若收到的项目中的距离d小于路由表中的距离,则更新

ELSE

不做任何事

③ 若180秒(RIP默认超时时间)还没有收到相邻路由器的更新路由表,

则将此相邻路由器记为不可达的路由器,则把距离设置为16(表示不可达)

④ 返回

① 有没有发现新网络?

② 旧网络的距离是否更新?

要么走老路,但距离变化

要么找一条新路且距离更近

例如。

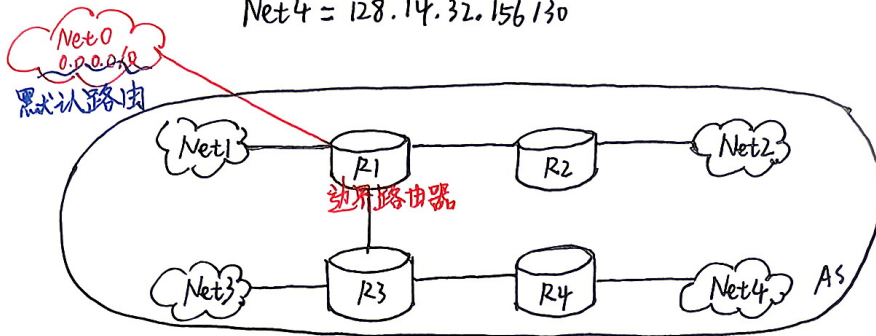
某地ISP构建了自身的自治系统,向上一级ISP购买了1个CIDR地址块并将其拆分成4个网络Net1 ~ Net4, 它们都用CIDR记法表示各自的网络前缀

Net1 = 128.14.32.128/28

Net2 = 128.14.32.144/29

Net3 = 128.14.32.152/30

Net4 = 128.14.32.156/30



规定该AS内使用的内部网关协议是RIP协议

假设路由器都同时开机,构建初始路由表

R1的初始路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付

R2的初始路由表

目的网络	距离	下一跳
Net2	1	直接交付

R3的初始路由表

目的网络	距离	下一跳
Net3	1	直接交付

R4的初始路由表

目的网络	距离	下一跳
Net4	1	直接交付

0时刻: 各路由器向邻居发送包含完整路由表的RIP响应报文

收到邻居的RIP报文后,更新自己的路由信息

(每个路由器仅与相邻路由器周期性地(30s)交换并更新路由信息)

R1的0时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	2	R2
Net3	2	R3

R2的0时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	1	直接交付

R3的0时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net3	1	直接交付
Net4	2	R4

R4的0时刻路由表

目的网络	距离	下一跳
Net3	2	R3
Net4	1	直接交付

30s时刻: 收到邻居的RIP报文后,更新自己的路由信息

R1的30时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	2	R2
Net3	2	R3
Net4	3	R3

R2的30时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	1	直接交付
Net3	3	R1

R3的30时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	3	R1
Net3	1	直接交付
Net4	2	R4

R4的30时刻路由表

目的网络	距离	下一跳
Net0	3	R3
Net1	3	R3
Net3	2	R3
Net4	1	直接交付

60s时刻: 经过本轮更新, RIP已收敛, 每台路由器都知道到达目的网络的最佳路由

R1的60时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	2	R2
Net3	2	R3
Net4	3	R3

R2的60时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	1	直接交付
Net3	3	R1
Net4	4	R1

R3的60时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	3	R1
Net3	1	直接交付
Net4	2	R4

R4的60时刻路由表

目的网络	距离	下一跳
Net0	3	R3
Net1	3	R3
Net2	4	R3
Net3	2	R3
Net4	1	直接交付

RIP的工作过程示例(动态适应网络拓扑变化)

接上例.

70s时刻 采用触发更新机制可以提升收敛速度

假如在R1、R4间新增一条链路

90s时刻:

R1的90时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	2	R2
Net3	2	R3
Net4	2	R4

R2的90时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	1	直接交付
Net3	3	R1
Net4	4	R1

R3的90时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	3	R1
Net3	1	直接交付
Net4	2	R4

R4的90时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	3	R1
Net3	2	R3
Net4	1	直接交付

120s时刻:

R1的120时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	2	R2
Net3	2	R3
Net4	2	R4

R2的120时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	1	直接交付
Net3	3	R1
Net4	3	R1

R3的120时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	3	R1
Net3	1	直接交付
Net4	2	R4

R4的120时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	3	R1
Net3	2	R3
Net4	1	直接交付

RIP 的缺点

- ① RIP 限制了网络的规模, 它能使用的最大距离为 15 (16 表示不可达)。
- ② 路由器之间交换的是路由器中的完整路由表, 网络规模越大, 开销越大
- ③ 当网络出现故障时, 路由器之间需反复多次交换信息才能完成收敛, 要经过较长时间才能将故障消息传送到所有路由器, 即“慢收敛现象”, 俗称“坏消息传播得慢”

距离向量路由可能会出现环路的情况, RIP 规定路径上的最高跳数 = 16, 这是一种防止环路的简单粗暴的方式。

接上例。

假如 1305 时刻 R2 路由器损坏

(R1 最近一次收到 R2 的 RIP 报文是在 1205 时刻, 因此应在 $120 + 180 = 3005$ 时刻判定 R2 不可达)

3005 时刻: R1 判定 R2 不可达 (认为 $R1 \rightarrow R2$ 的距离 = 16) 并更新自己的路由表

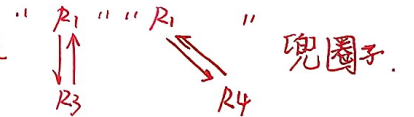
R1 的 300 时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	4	R2/R4 ✗
Net3	2	R3
Net4	2	R4

R1 误以为可以从 R3/R4 到达 Net2

RIP 协议支持多条等价路径共存, 将它们同时加入路由表, 实现负载均衡转发 (例如轮询转发)。

此时, 转发至 Net2 的路径形成环路, IP 数据报在 “兜圈子” 直至 TTL=0 才会被丢弃



各路由器向邻居发送 RIP 报文 (包含完整路由表)

R1 的 300 时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	4	R3/R4 ✗
Net3	2	R3
Net4	2	R4

R3 的 300 时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	4	R4 ✗
Net3	1	直接交付
Net4	2	R4

R4 的 300 时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	4	R3 ✗
Net3	2	R3
Net4	1	直接交付

330s时刻:

R1的330时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	5	R3/R4
Net3	2	R3
Net4	2	R4

R3的330时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	5	R1/R4
Net3	1	直接交付
Net4	2	R4

R4的330时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	5	R1/R3
Net3	2	R3
Net4	1	直接交付

每经过一轮路由信息交换, 到达Net2的距离+1.

多轮迭代后Net2的距离增加到16, 才能确定Net2不可达

$$16 - 5 = 11 (\text{轮})$$

$$11 \times 30 = 330 (s).$$

$$330 + 330 = 660 (s)$$

660s时刻: 收敛

R1的660时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	16	NULL
Net3	2	R3
Net4	2	R4

R3的660时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	16	NULL
Net3	1	直接交付
Net4	2	R4

R4的660时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	16	NULL
Net3	2	R3
Net4	1	直接交付

RIP的优点:

- ① 实现简单、开销小、收敛过程较快
- ② 若一个路由器发现了更短的路由, 则这种更新信息传播得很快, 在较短时间内便可被传至所有路由器

俗称“好消息传播得快”

接上例.

670s时刻: R2恢复

- 路由器刚开始工作时, 只知道自己到直接相连的几个网络的距离是1
- 新路由器构建好初始路由表, 就可以立即发送给邻居
- 新路由器, 可以用RIP请求报文, 迅速向邻居请求路由信息
(R2向R1发送RIP请求报文, R1回给R2 RIP响应报文)

R1的670时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	16	NULL R2
Net3	2	R3
Net4	2	R4

R2的670时刻路由表

目的网络	距离	下一跳
Net2	1	直接交付
Net0	2	R1
Net1	2	R1
Net2	1	直接交付
Net3	3	R1
Net4	3	R1

R3的670时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	16	NULL
Net3	1	直接交付
Net4	2	R4

R4的670时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	16	NULL
Net3	2	R3
Net4	1	直接交付

新Router R2 迅速收敛

690s时刻: R1、R3、R4 定时发送RIP报文, R2是670s时刻开机, 要到700s时刻才发下次RIP报文

R1的690时刻路由表

目的网络	距离	下一跳
Net0	1	外部路由
Net1	1	直接交付
Net2	2	R2
Net3	2	R3
Net4	2	R4

R2的690时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	1	直接交付
Net3	2	R3 R1
Net4	3	R1

R3的690时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	3	R1
Net3	1	直接交付
Net4	2	R4

R4的690时刻路由表

目的网络	距离	下一跳
Net0	2	R1
Net1	2	R1
Net2	3	R1
Net3	2	R3
Net4	1	直接交付

所有路由器迅速收敛

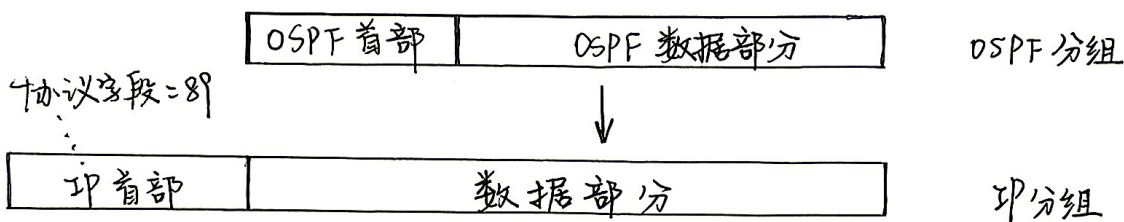
开放最短路径优先协议

OSPF在协议栈中所处的位置

- OSPF 属于网络层
- OSPF 使用IP协议提供的服务, IP首部的协议字段=89
- OSPF的PDU常译为"OSPF分组"或"OSPF数据报"
- OSPF协议定义了5种OSPF分组类型:

在OSPF首部中指明

Type 值	英文缩写	中文译名
1	Hello Packet	问候分组
2	DD Packet	数据库描述分组
3	LSR Packet	链路状态请求分组
4	LSU Packet	链路状态更新分组
5	LSAck Packet	链路状态确认分组



OSPF 的特点

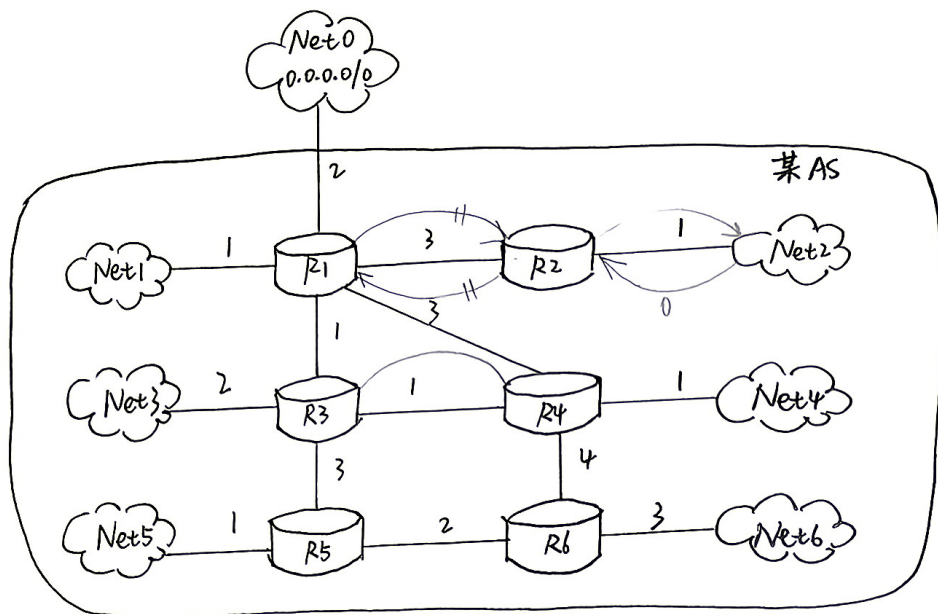
OSPF属于网络层,使用IP协议提供的服务,IP首部的协议字段=89
OSPF基于链路状态路由算法(Dijkstra算法)计算最佳转发路径
每一台路由器都需要建立整个网络的拓扑图(带权有向图,可用邻接表存储)

OSPF工作流程 (路由器视角)

- 路由器开机时探测与自己直连的所有链路状态(LS, Link-State)
- 用洪泛法(flooding)将探测到的链路状态信息迅速发给其它路由器(每个链路状态信息都带上一个32位的序号,序号越大,状态越新)
- 若路由器探测到身边链路有变化,就再次立即洪泛最新的链路状态信息

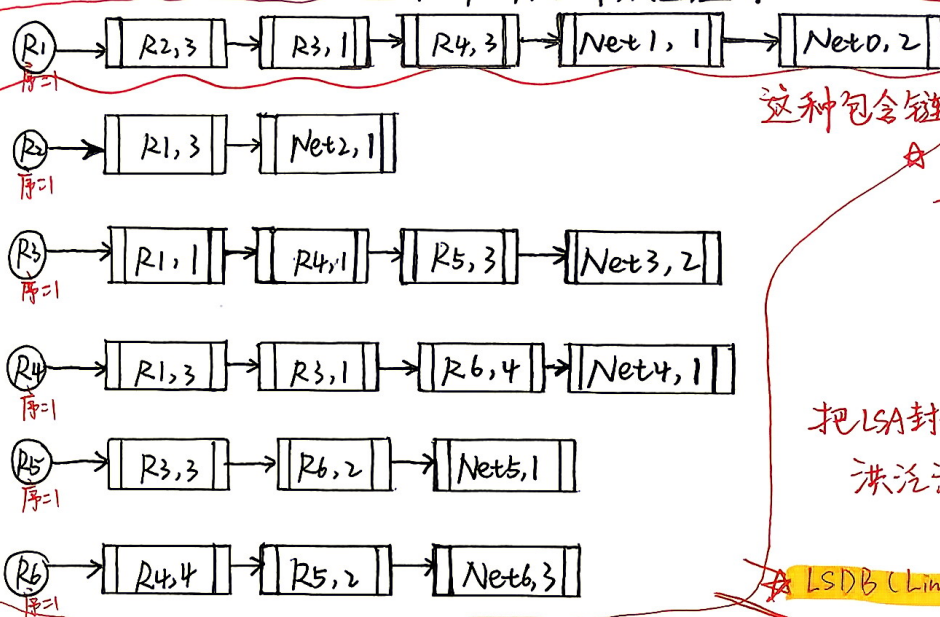
其他

- OSPF允许自治系统灵活地自定义链路“代价”
- OSPF支持等价多路径转发
- OSPF分组支持鉴别功能,防止非法路由信息在自治系统内传播
- OSPF支持变长子网划分、CIDR



★ LSI (Link State Information):
链路状态信息, 广义的概念.
指代所有描述链路状态的信息,
可小可大

路由器可以直接探测到的信息: 自己和哪些节点直连?



这种包含链路状态信息的单链表:

★ LSA (Link State Advertisement):
链路状态通告

一种
数据结构:

采用带权有向图的
邻接表存储

把LSA封装到OSPF分组中, 随后采用
洪泛法迅速覆盖全网

★ LSDB (Link-State Database) = 链路状态数据库

怎么把自己探测到的信息说给其它所有路由器?



What - 说什么: 与自己直连的所有链路状态 (LS, Link-State)

我是谁?

我邻居有哪些人?

我到每个邻居之间的代价?

数据结构 图的顶点 (Net6) → [R6, 0]

带权有向边

How - 怎么说: 洪泛法 (flooding)

- 传十, 十传百 ... 迅速“广而告之”

防止无休止的洪泛转发 { 洪泛信息不能“回流转发” (转发到除入口外所有出口)
若收到重复的链路状态信息, 不可重复转发

When - 何时说: 探测到身边有变化

路由器探测到与自己直连的链路或节点发生变化时, 立即洪泛最新链路状态信息

不存在“坏消息传播得慢”的问题!

OSPF 其它特点:

- ① OSPF 允许对每条路由设置成不同的代价, 对于不同类型的业务可计算出不同的路由

例如, OSPF 默认基于带宽计算链路代价: $\text{metric} = \frac{\text{参考带宽}}{\text{接口带宽}}$

(参考带宽默认为 100Mbps)

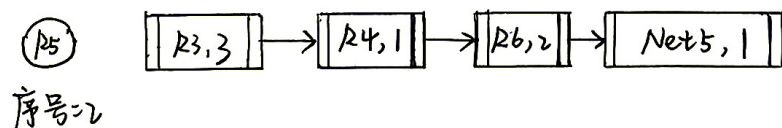
常为简化, 默认两台路由器之间的链路的两个方向权值相同.

现实应用中.

{ 一段链路的两个方向权值可能不同.
"直连网络 → 路由器" 方向的权值通常是 0.

- ② 若到同一目的网络有多条相同代价的路径, 可将通信量分配给这几条路径, 使负载均衡
- ③ OSPF 分组具有鉴别功能, 可保证仅在可信赖的路由器之间交换链路状态信息.
一来, 可以防止非法路由器的恶意攻击 (避免其利用 OSPF 洪泛伪造链路状态信息);
二来, 可以防止中间人攻击篡改 OSPF 洪泛分组
- ④ OSPF 支持可变长度子网划分和无分类编址 CIDR
- ⑤ 每个链路状态都带上 1 个 32 位序号. 序号越大, 状态越新.

若在 R4、R5 中间加一根线, 则:



OSPF的工作原理

OSPF的工作原理

如何生成路由表

① 构建LSDB

路由器用洪泛法将探测到的链路状态信息迅速发给其它路由器。一旦探测到身边链路有变化,就再次立即洪泛最新的链路状态信息。

各路由器根据其他路由器发来的链路状态信息构建LSDB (链路状态数据库,本质是图的邻接表)

② 运行Dijkstra算法

路由器基于LSDB运行Dijkstra算法,计算出从自身到达每一个目的网络的最短路径

③ 生成路由表

根据Dijkstra算法的运行结果,构造路由表

路由表结构为 [目的网络, 下一条, 到目的网络的距离]

划分区域

将自治系统划分为一个主干区域和多个非主干区域

作用

将“洪泛”的范围限制在每个区域内部,降低洪泛压力

一台普通路由器仅关心所属区域的网络拓扑,即LSDB变小

哪几类?

自治系统路由器 (ASBR): 主干区域内至少有一台与其它AS相连

区域边界路由器 (ABR): 每个非主干区域至少有一台与主干区域相连

区域内部路由器 (Internal Router): 亦称“非边界路由器”,通常每个区域内有多台

接上例.

R1 运行 Dijkstra 算法后可知:

到 Net0 的最短路径: Cost=2, R1 → Net0

到 Net1 的最短路径: Cost=1, R1 → Net1

到 Net2 的最短路径: Cost=4, R1 → R2 → Net2

到 Net3 的最短路径: Cost=3, R1 → R3 → Net3

到 Net4 的最短路径: Cost=3, R1 → R3 → R4 → Net4

到 Net5 的最短路径: Cost=5, R1 → R3 → R5 → Net5

到 Net6 的最短路径: Cost=9, R1 → R3 → R5 → R6 → Net6
或 R1 → R3 → R4 → R6 → Net6

改进版 Dijkstra 算法支持找到多条等价最短路径

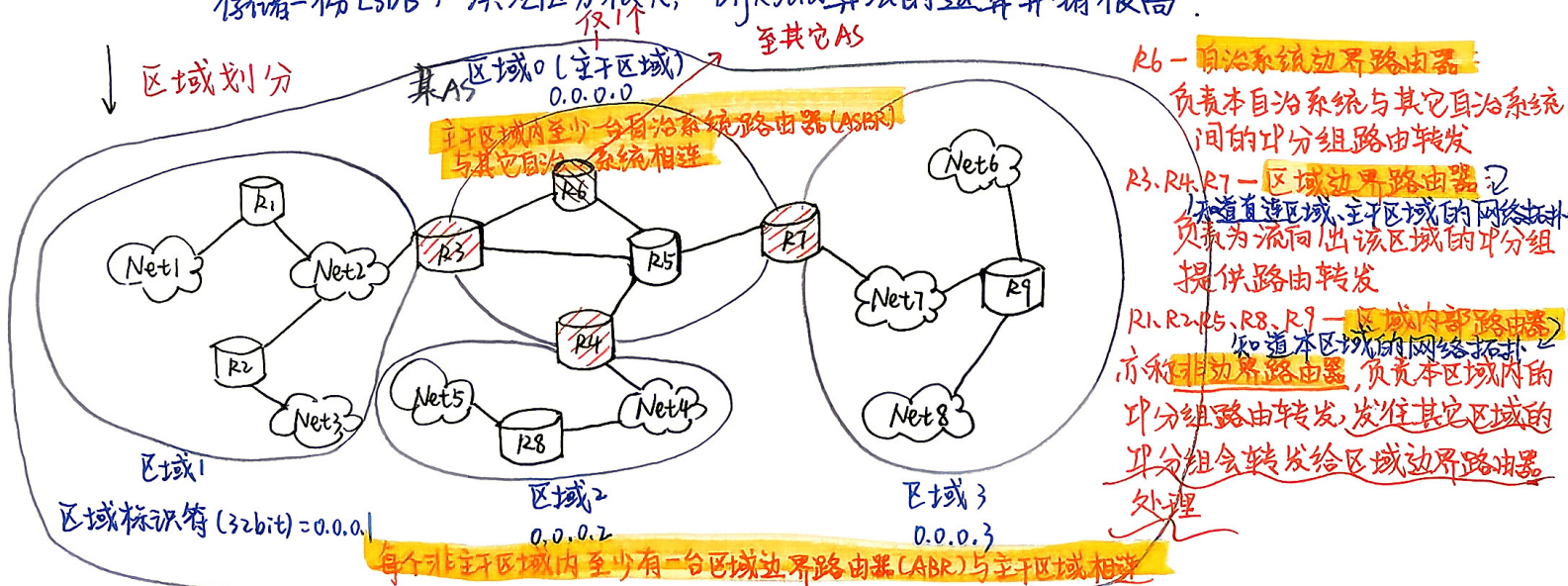
由 IP 地址、子网掩码组成 (例如 168.168.0.0/126)

R1 生成的路由表

目的网络	下一跳	距离
Net1	直接交付	1
Net2	R2	4
Net3	R3	3
Net4	R3	3
Net5	R3	5
Net6	R3	9
Net0	外部路由器	2

- 虽然使用 Dijkstra 算法可计算出完整最优路径, 但路由表中不会存储完整路径而只存储“下一跳”
- 若网络拓扑发生变化, 则立即引起洪泛, 从而引起各台路由器的 LSDB 变化, 并再次运行 Dijkstra 算法, 构造新路由表

但是, 如果自治系统规模很大, 路由器、网络很多, LSDB 很复杂且每台路由器都会存储一份 LSDB, 洪泛压力很大, Dijkstra 算法的运算开销很高.

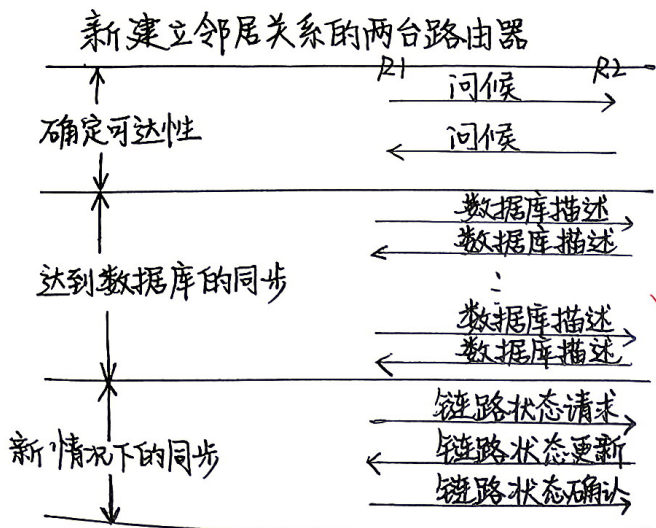


为使 OSPF 能用于规模很大的网络, 可将一个 AS 再划分为若干区域 (Area), 使洪泛法交换链路状态信息的范围局限在各个区域而非整个 AS.

这样可以减少网络通信量, 也让每个路由器的 LSDB 更小

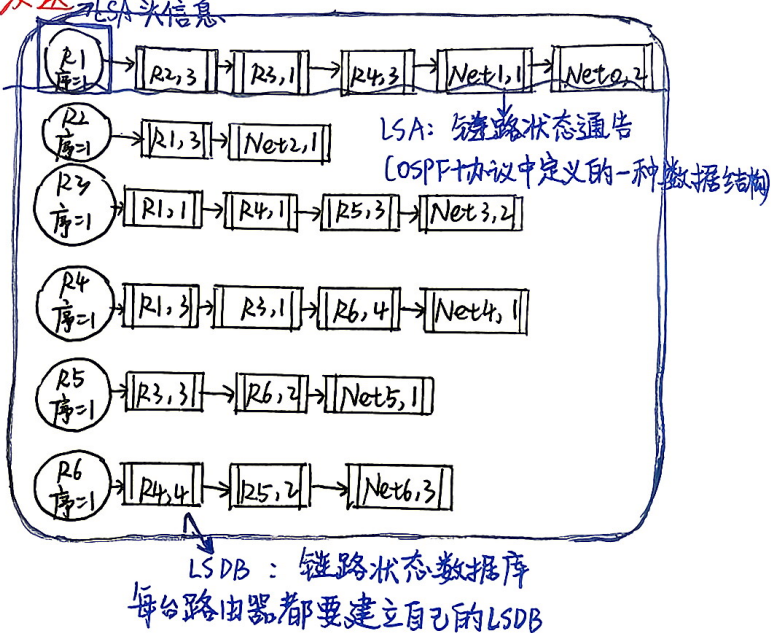
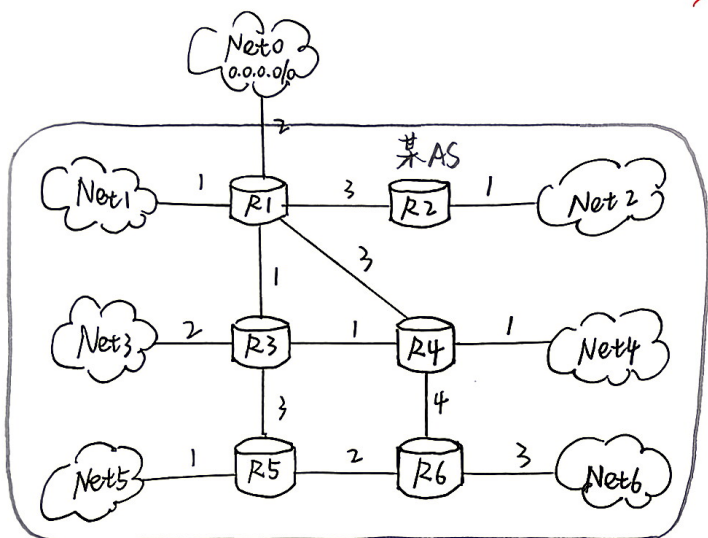
在一个 OSPF 自治系统内, 只有一个主干区域 (而非主干区域常与主干区域直接相连), 其标识符 Area ID=0, 作用是连通其他区域.

OSPF 的分组类型



Type 值	英文缩写	中文译名	作用
1	Hello Packet	问候分组	建立维持邻居关系(每隔10s发一次,40s超时)
2	DD Packet	数据库描述分组	邻居建立时,向邻居给出自己的LSDB摘要(即LSA头信息)
3	LSR Packet	链路状态请求分组	向邻居请求缺少的LSA(指明头信息即可)
4	LSU Packet	链路状态更新分组	向邻居传输具体的LSA(可能引发全网洪泛)
5	LSAck Packet	链路状态确认分组	收到邻居发来的LSU后,向邻居确认收到了哪些LSA(头信息)

由于IP数据报的数据部分长度有限,如果LSDB摘要中的信息过长,就要拆分成多个数据库描述分组,即通过多个OSPF分组封装到多个IP数据报中去发送。



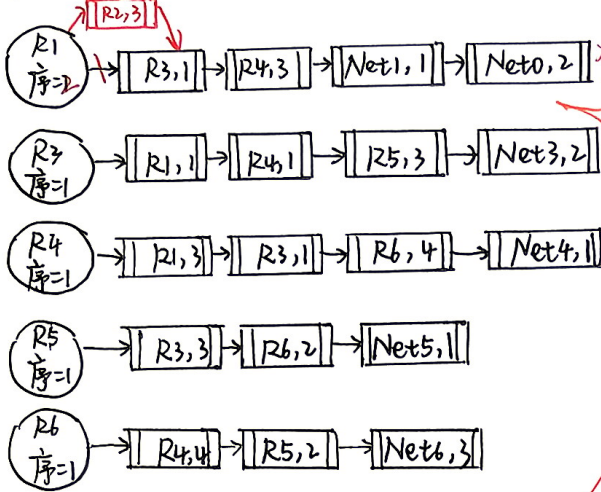
Type 1: Hello 分组 (问候分组)

各路由器每隔10秒向直接邻居发送一次“问候分组”。
若超过40秒未收到邻居的问候就认为该邻居不可达

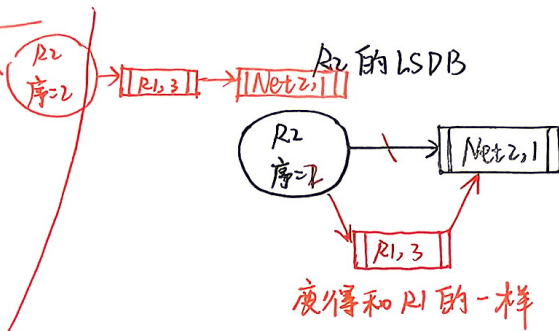
假设某时刻 R2 与 R1 建立连接

- 向新邻居发送 Hello 分组
- 确认邻居关系

R1 的 LSDB

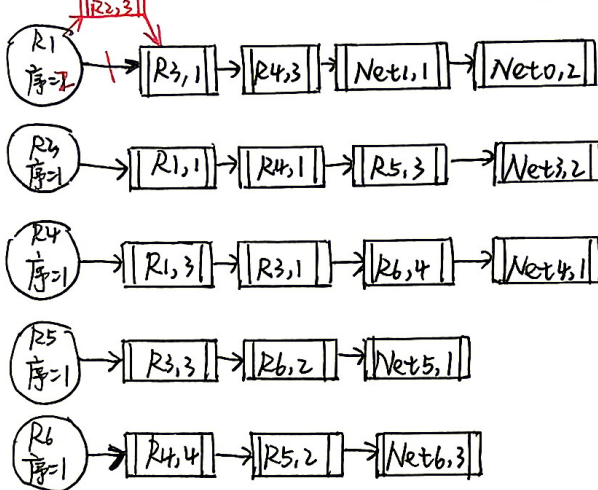


将探测到的新信息通过 LSU 立即洪泛通知全网



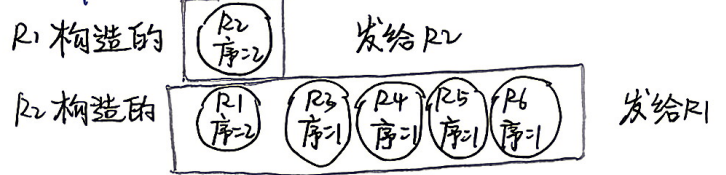
- 更新链路状态信息

R3, R4, R5, R6 的 LSDB



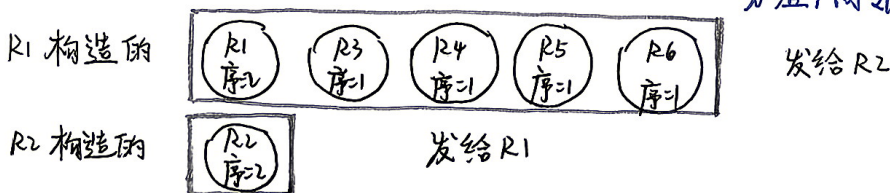
Type 5: LSAck 分组 (链路状态确认分组)

路由器收到邻居发来的 LSU 后返回 LSAck 分组, 向邻居确认收到了哪些 LSA (头信息)



Type 2: DD 分组 (数据库描述分组)

两台路由器的邻居关系建立时, 通过数据库描述分组, 向邻居给出自己的 LSDB 摘要 (即 LSA 头信息)



Type 3: LSR 分组 (链路状态请求分组)

通过 LSR 分组向邻居请求缺少的 LSA (头信息)



另外注意, 即便重复收到洪泛的 LSU 分组 (例如 R4 收到 R1 发来的, 也会收到 R3 发来的) 也要给邻居返回 LSAck 既保证可靠性, 又不会无休止地重复。

Type 4: LSU 分组 (链路状态更新分组)

路由器通过 LSU 分组向邻居传输具体 LSA (可能引发全网洪泛)

R1 构造包含 R2 请求的所有 LSA 完整信息的 LSU 分组发给 R2

R1 收到 R2 发来的 LSU 分组获得新链路的状态信息, 需洪泛至全网。

R2 构造包含 R1 请求的所有 LSA 完整信息的 LSU 分组发给 R1

最后 R1~R6 的 LSDB 都一样